

```

// File: Assets/_Scripts/World/WeatherController.cs
//
// QUAHOG – Region-driven weather finite state machine.
// Single source of truth for environmental state. Gameplay systems SUBSCRIBE to
// broadcast (PedestrianAI -> WeatherPanic, post-processing -> fog) rather than p
// and vehicle physics reads CurrentFriction directly. Slick roads, fog density,
// "isStorming" flag all travel with the event so nothing else has to know about
//
// Matches the API the existing blueprints already call:
//   WeatherController.OnWeatherChanged += HandleWeatherShift;    // static, no sc
//   void HandleWeatherShift(WeatherModifiers w) { if (w.isStorming) ... }
//
// Design: weighted random transitions on a per-state timer; continuous values (f
// fog) blend over `blendSeconds` so tires/visuals don't pop, while the discrete
// edge fires immediately (so AI reacts the instant a storm rolls in).

using System;
using System.Collections.Generic;
using UnityEngine;

public enum WeatherState { Clear, DenseFog, CoastalRain, Noreaster }

/// <summary>Lightweight value type passed by the event – a struct, so broadcasts
public struct WeatherModifiers
{
    public WeatherState state;
    public float frictionMultiplier; // 1.0 = dry grip, <1.0 = slick. Vehicles s
    public float fogDensity;         // 0..1 -> drives the volumetric fog volume
    public bool isStorming;          // true for CoastalRain / Noreaster (umbrel

    public WeatherModifiers(WeatherState state, float friction, float fog, bool s
    {
        this.state = state;
        frictionMultiplier = friction;
        fogDensity = fog;
        isStorming = storming;
    }
}

[DisallowMultipleComponent]
public class WeatherController : MonoBehaviour
{
    public static WeatherController Instance { get; private set; }

```

```

/// <summary>
/// Fires once per state change. Static so AI/vehicle prefabs can subscribe i
/// without holding a scene reference.
/// CONTRACT: every subscriber MUST unsubscribe in OnDisable. A destroyed ped
/// stays subscribed will throw on the next broadcast and leak – this is the
/// with static events, so the spawner/pooling layer has to honor it.
/// </summary>
public static event Action<WeatherModifiers> OnWeatherChanged;

[Serializable]
public class WeatherProfile
{
    public WeatherState state;
    [Range(0f, 1f)] public float frictionMultiplier = 1f;
    [Range(0f, 1f)] public float fogDensity = 0f;
    public bool isStorming = false;

    [Tooltip("Relative chance this state is chosen next. 0 = never auto-select")]
    public float selectionWeight = 1f;

    [Tooltip("How long this state holds, in seconds (min, max).")]
    public Vector2 durationRange = new Vector2(120f, 300f);
}

[Header("Weather Profiles – add one entry per WeatherState")]
[SerializeField]
private List<WeatherProfile> profiles = new List<WeatherProfile>
{
    // Sensible South Coast defaults; tune in the Inspector.
    new WeatherProfile { state = WeatherState.Clear,          frictionMultiplier
    new WeatherProfile { state = WeatherState.DenseFog,       frictionMultiplier
    new WeatherProfile { state = WeatherState.CoastalRain,     frictionMultiplier
    new WeatherProfile { state = WeatherState.Noreaster,       frictionMultiplier
};

[Header("Transition")]
[Tooltip("Seconds to lerp friction/fog from the old state into the new one.")]
[SerializeField] private float blendSeconds = 6f;
[SerializeField] private WeatherState startingState = WeatherState.Clear;

[Header("Debug")]
[SerializeField] private bool logTransitions = false;

/// <summary>Latest fully-resolved modifiers. Continuous consumers (fog, tire
public WeatherModifiers Current { get; private set; }
public WeatherState CurrentState => _active != null ? _active.state : Weather
public float CurrentFriction => Current.frictionMultiplier;

```

```

public float CurrentFog => Current.fogDensity;
public bool IsStorming => Current.isStorming;

private WeatherProfile _active;
private float _stateTimer;
private float _stateDuration;
private float _totalWeight;

// Blend bookkeeping for the continuous values.
private float _blendT = 1f;
private float _fromFriction, _fromFog, _toFriction, _toFog;

private void Awake()
{
    // Strict singleton - duplicate currency/weather states are how save corr
    if (Instance != null && Instance != this) { Destroy(gameObject); return;
    Instance = this;

    _totalWeight = 0f;
    for (int i = 0; i < profiles.Count; i++)
        _totalWeight += Mathf.Max(0f, profiles[i].selectionWeight);

    EnterState(GetProfile(startingState), instant: true);
}

private void OnDestroy()
{
    if (Instance == this) Instance = null;
}

private void Update()
{
    // Blend the CONTINUOUS values after a switch so grip/fog don't snap.
    // (The discrete state edge already broadcast in EnterState.)
    if (_blendT < 1f && blendSeconds > 0f)
    {
        _blendT = Mathf.Min(1f, _blendT + Time.deltaTime / blendSeconds);
        float fr = Mathf.Lerp(_fromFriction, _toFriction, _blendT);
        float fog = Mathf.Lerp(_fromFog, _toFog, _blendT);
        Current = new WeatherModifiers(_active.state, fr, fog, _active.isStor
    }

    _stateTimer += Time.deltaTime;
    if (_stateTimer >= _stateDuration)
        EnterState(PickNextWeighted());
}

```

```

private void EnterState(WeatherProfile next, bool instant = false)
{
    if (next == null) return; // misconfigured profiles list - fail safe, kee

    _active = next;
    _stateTimer = 0f;
    _stateDuration = UnityEngine.Random.Range(next.durationRange.x, next.dura

    _fromFriction = instant ? next.frictionMultiplier : Current.frictionMulti
    _fromFog      = instant ? next.fogDensity          : Current.fogDensity;
    _toFriction   = next.frictionMultiplier;
    _toFog        = next.fogDensity;
    _blendT       = instant ? 1f : 0f;

    Current = new WeatherModifiers(next.state, _fromFriction, _fromFog, next.

    if (logTransitions) Debug.Log($"[Weather] -> {next.state} (friction {next

    // Discrete broadcast on the state EDGE. AI flips behaviour (e.g. Weather
    OnWeatherChanged?.Invoke(Current);
}

/// <summary>
/// Designer / mission hook. Use this to drive scripted weather - e.g. "The G
/// forces Noreaster, then ClearForced() (or just let the timer resume) after
/// </summary>
public void ForceState(WeatherState state) => EnterState(GetProfile(state));

private WeatherProfile GetProfile(WeatherState state)
{
    for (int i = 0; i < profiles.Count; i++)
        if (profiles[i].state == state) return profiles[i];
    return profiles.Count > 0 ? profiles[0] : null; // fallback avoids a null
}

private WeatherProfile PickNextWeighted()
{
    if (_totalWeight <= 0f || profiles.Count == 0) return _active; // nothing
    float roll = UnityEngine.Random.value * _totalWeight;
    for (int i = 0; i < profiles.Count; i++)
    {
        roll -= Mathf.Max(0f, profiles[i].selectionWeight);
        if (roll <= 0f) return profiles[i];
    }
    return profiles[profiles.Count - 1]; // float rounding guard
}
}

```

